

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 10 – Find Max/Min



HAIDAR RAFI ARSHIA
109082530022
KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

Pencarian nilai ekstrim adalah cara dasar untuk mencari nilai terbesar atau terkecil dalam sekumpulan data yang disusun berurutan. Ide dasarnya sangat sederhana, yaitu menganggap elemen pertama sebagai nilai ekstrim sementara, lalu menyimpannya dalam sebuah variabel untuk dibandingkan dengan elemen-elemen lainnya.

Dalam bahasa Go, proses ini menggunakan perulangan dan percabangan untuk memvalidasi nilai. Jika ditemukan data yang lebih ekstrim, variabel penyimpan diperbarui. Algoritma ini juga menyimpan indeks elemen untuk memudahkan pelacakan lokasi data.

Algoritma ini sangat fleksibel karena bisa diterapkan pada berbagai tipe data, dari integer dan float hingga tipe data terstruktur seperti struct. Pada tipe data terstruktur, pencarian dilakukan berdasarkan salah satu field, tetapi masih bisa mengembalikan informasi lengkap objek tersebut melalui indeks yang ditemukan. Ini membuat algoritma pencarian ekstrim penting dalam pengolahan data yang lebih kompleks.

B. Guided

1. Guided1.go

```
1 package main
2
3 import "fmt"
4
5 func nilaiTerbesar(array []int) (int, int) {
6     var idx int
7     var nilaiTerbesar int = array[0]
8     var idxDitemukan int = 0
9     for idx = 1; idx < len(array); idx++ {
10         if array[idx] > nilaiTerbesar {
11             nilaiTerbesar = array[idx]
12             idxDitemukan = idx
13         }
14     }
15     return nilaiTerbesar, idxDitemukan
16 }
17
18 func nilaiTerkecil(array []int) (int, int) {
19     var idx int
20     var nilaiTerkecil int = array[0]
21     var idxDitemukan int = 0
22     for idx = 1; idx < len(array); idx++ {
23         if array[idx] < nilaiTerkecil {
24             nilaiTerkecil = array[idx]
25             idxDitemukan = idx
26         }
27     }
28     return nilaiTerkecil, idxDitemukan
29 }
30
31 func main() {
32     var arrAngka [5]int = [5]int{45, 23, 98, 66, 12}
33
34     var nilaiMaks, idxNilaiMaks int = nilaiTerbesar(arrAngka[:])
35     var nilaiMin, idxNilaiMin int = nilaiTerkecil(arrAngka[:])
36
37     fmt.Printf("Nilai terkecil : %d ditemukan pada index ke-%d", nilaiMin, idxNilaiMin)
38     fmt.Println()
39     fmt.Printf("Nilai terbesar : %d ditemukan pada index ke-%d", nilaiMaks, idxNilaiMaks)
40
41 }
```

Output:

```
● PS D:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia> go run
"
    Nilai terkecil : 12 ditemukan pada index ke-4
    Nilai terbesar : 98 ditemukan pada index ke-2
● PS D:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia> go run
"
    Nilai terkecil : 12 ditemukan pada index ke-4
    Nilai terbesar : 98 ditemukan pada index ke-2
○ PS D:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia>
```

Penjelasan:

Program ini mengimplementasikan algoritma pencarian nilai ekstrim (maksimum dan minimum) dengan menetapkan elemen pertama array sebagai acuan awal, kemudian melakukan validasi melalui perulangan sekuensial untuk membandingkan data berikutnya hingga akhir array. Selain mengingat nilai terbesar dan terkecil, kode ini secara spesifik menyimpan posisi atau **indeks yang ditemukan** dari nilai tersebut, yang merupakan prinsip penting dalam pencarian data terstruktur agar identitas atau lokasi data dapat diketahui kembali. Penggunaan fungsi yang mengembalikan dua nilai sekaligus (return nilai, idx) menunjukkan efisiensi dalam memproses array untuk mendapatkan hasil pencarian yang komprehensif sesuai panduan pada modul 10.

2. Guided2.go

```
1 package main
2
3 import "fmt"
4
5 type mahasiswa struct {
6     nama string
7     nim string
8     IPK float64
9 }
10
11 type arrMahasiswa [3]mahasiswa
12
13 func IPKTerdasar(array []mahasiswa) (mahasiswa, int) {
14     var mhsTerdasar mahasiswa = array[0]
15     var idxDitemukan int = 0
16     var i int
17
18     for i = 1; i < len(array); i++ {
19         if array[i].IPK > mhsTerdasar.IPK {
20             mhsTerdasar = array[i]
21             idxDitemukan = i
22         }
23     }
24
25     return mhsTerdasar, idxDitemukan
26 }
27
28 func IPKTerdasar(array []mahasiswa) (mahasiswa, int) {
29     var mhsTerdasar mahasiswa = array[0]
30     var idxDitemukan int = 0
31     var i int
32
33     for i = 1; i < len(array); i++ {
34         if array[i].IPK < mhsTerdasar.IPK {
35             mhsTerdasar = array[i]
36             idxDitemukan = i
37         }
38     }
39     return mhsTerdasar, idxDitemukan
40 }
41
42 func main() {
43     var arrMhs arrMahasiswa
44     var i, j int
45     for i = 0; i < len(arrMhs); i++ {
46         fmt.Println("DATA MAHASISWA INDEKS KE-", i)
47         fmt.Print("Masukkan nama : ")
48         fmt.Scan(&arrMhs[i].nama)
49         fmt.Print("Masukkan NIM : ")
50         fmt.Scan(&arrMhs[i].nim)
51         fmt.Print("Masukkan IPK : ")
52         fmt.Scan(&arrMhs[i].IPK)
53         fmt.Println("=====")
54     }
55
56     fmt.Println()
57     for j = 0; j < len(arrMhs); j++ {
58         fmt.Println("DATA MAHASISWA INDEKS KE-", j)
59         fmt.Println("Nama : ", arrMhs[j].nama)
60         fmt.Println("NIM : ", arrMhs[j].nim)
61         fmt.Println("IPK : ", arrMhs[j].IPK)
62         fmt.Println("=====")
63     }
64     fmt.Println()
65
66     var mhsMaks, mhsMin mahasiswa
67     var idxMaks, idxMin int
68
69     mhsMaks, idxMaks = IPKTerdasar(arrMhs[:])
70     mhsMin, idxMin = IPKTerdasar(arrMhs[:])
71
72     fmt.Println("=== DATA IPK TERKECIL & TERBESAR MAHASISWA ===")
73     fmt.Printf("IPK Terbesar : %.2f, atas nama %s dengan NIM %s, ditemukan pada indeks ke-%d", mhsMaks.IPK, mhsMaks.nama, mhsMaks.nim, idxMaks)
74     fmt.Printf("IPK Terkecil : %.2f, atas nama %s dengan NIM %s, ditemukan pada indeks ke-%d", mhsMin.IPK, mhsMin.nama, mhsMin.nim, idxMin)
75 }
```

Output :

```
PS D:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia> go run "d:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia\Week9_Modul19\Guided\No2.go"
"
DATA MAHASISWA INDEKS KE- 2
Masukkan nama : Hai
Masukkan NIM :109082530022
Masukkan IPK : 3.5
=====
DATA MAHASISWA INDEKS KE- 0
Nama : Haidar
NIM : Rafi
IPK : 0
=====
DATA MAHASISWA INDEKS KE- 1
Nama : rshia
NIM : 109082530022
IPK : 3.9
=====
DATA MAHASISWA INDEKS KE- 2
Nama : Hai
NIM : 109082530022
IPK : 3.5
=====
=== DATA IPK TERKECIL & TERBESAR MAHASISWA ===
IPK Terbesar : 3.90, atas nama rshia dengan NIM 109082530022, ditemukan pada indeks ke-1IPK Terkecil : 0.00, atas nama Haidar dengan NIM Rafi, ditemukan pada indeks ke-0
PS D:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia>
```

Penjelasan:

Program ini mengimplementasikan pencarian nilai ekstrim pada **tipe data terstruktur (struct)** untuk mengelola data siswa, di mana perbandingan dilakukan berdasarkan lapangan IPK namun tetap mempertahankan keutuhan seluruh identitas siswa tersebut. Berikut konsep pada modul 10.3, fungsi `IPKTerbesar` dan `IPKTerkecil` menggunakan elemen pertama array sebagai nilai acuan untuk divalidasi secara sekuensial terhadap data siswa lainnya. Dengan mengembalikan variabel bertipe `struct` dan nilai **indeks**, program dapat secara efisien menampilkan informasi lengkap (nama dan NIM) siswa dengan IPK tertinggi atau terendah tanpa kehilangan jejak lokasi di dalam array.

C. Unguided

1. Soal Unguided 1

ProgramTariflkan.go

Jawaban :

```
1  package main
2
3  import "fmt"
4
5  func main() {
6      var n int
7      var berat [1000]float64
8
9      fmt.Print("Masukkan jumlah anak kelinci (N): ")
10     fmt.Scan(&n)
11
12     for i := 0; i < n; i++ {
13         fmt.Scan(&berat[i])
14     }
15
16     min := berat[0]
17     max := berat[0]
18
19     for i := 1; i < n; i++ {
20         if berat[i] < min {
21             min = berat[i]
22         }
23         if berat[i] > max {
24             max = berat[i]
25         }
26     }
27
28     fmt.Printf("Berat terkecil: %.2f\n", min)
29     fmt.Printf("Berat terbesar: %.2f\n", max)
30 }
```

Output :

```
PS D:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia> go run
go"
Masukkan jumlah anak kelinci (N): 10
7.5
25
6.7
5
2
3
9
8.7
6
4.5
Berat terkecil: 2.00
Berat terbesar: 25.00
PS D:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia> |
```

Penjelasan :

Program ini mengimplementasikan algoritma pencarian nilai ekstrim (minimum dan maksimum) pada sekelompok data nyata yang disimpan dalam sebuah **array** dengan kapasitas 1000. Sesuai dengan ide algoritma pada modul 10, program mengatur data pertama sebagai nilai ekstrim sementara, kemudian melakukan validasi dengan membandingkan data tersebut terhadap data kedua hingga data terakhir menggunakan struktur kontrol perulangan. Program ini menunjukkan cara melakukan pembaruan nilai variabel mindan maxsecara sekuensial untuk mendapatkan hasil akhir yang valid dari kumpulan data acak.

2. Soal Unguided 2

ProgramTarifIkan.go

Jawaban :

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     var x, y int
7     var beratIkan [1000]float64
8
9     fmt.Scan(&x, &y)
10
11     for i := 0; i < x; i++ {
12         fmt.Scan(&beratIkan[i])
13     }
14
15     var totalSeluruhWadah float64
16     var jumlahWadah int = 0
17
18     for i := 0; i < x; i += y {
19         var beratWadah float64 = 0
20
21         for j := i; j < i+y && j < x; j++ {
22             beratWadah += beratIkan[j]
23         }
24         fmt.Printf("%.2f ", beratWadah)
25         totalSeluruhWadah += beratWadah
26         jumlahWadah++
27     }
28
29     rataRata := totalSeluruhWadah / float64(jumlahWadah)
30     fmt.Printf("\nRata-rata berat per wadah: %.2f\n", rataRata)
31 }
```

Output :

```
PS D:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia> go run ramTarifIkan.go
12
4
5
7
8
3
5
4
20
1
3
5
7
8
23.00 30.00 23.00
25.33
PS D:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia>
```

Penjelasan :

Program ini menggunakan **array** untuk menampung data berat ikan dan melakukan pengukuran data berdasarkan pengelompokan (wadah). Program logika melibatkan manipulasi indeks array di mana data diproses dalam blok berukuran Y (kapasitas wadah) untuk menghitung total berat di setiap kelompoknya. Selain melakukan akumulasi berat, program ini juga menerapkan perhitungan statistik sederhana untuk mencari nilai rata-rata dari total berat per wadah yang telah dihasilkan, menunjukkan penerapan penggunaan array dalam menangani kasus distribusi data terstruktur.

3. Soal Unguided 3

ProgramPosyandu.go

Jawab:

```
1 package main
2
3 import "fmt"
4
5 type arrBalita [100]float64
6
7 func hitungMinMax(arrBerat arrBalita, n int, bMin, bMax *float64) {
8     *bMin = arrBerat[0]
9     *bMax = arrBerat[0]
10
11     for i := 1; i < n; i++ {
12         if arrBerat[i] < *bMin {
13             *bMin = arrBerat[i]
14         }
15         if arrBerat[i] > *bMax {
16             *bMax = arrBerat[i]
17         }
18     }
19 }
20
21 func rerata(arrBerat arrBalita, n int) float64 {
22     var total float64 = 0
23     for i := 0; i < n; i++ {
24         total += arrBerat[i]
25     }
26     return total / float64(n)
27 }
28
29 func main() {
30     var n int
31     var data arrBalita
32     var min, max float64
33
34     fmt.Print("Masukan banyak data berat balita: ")
35     fmt.Scan(&n)
36
37     for i := 0; i < n; i++ {
38         fmt.Printf("Masukan berat balita ke-%d: ", i+1)
39         fmt.Scan(&data[i])
40     }
```

```

41
42     hitungMinMax(data, n, &min, &max)
43
44     fmt.Printf("Berat balita minimum: %.2f kg\n", min)
45     fmt.Printf("Berat balita maksimum: %.2f kg\n", max)
46     fmt.Printf("Rerata berat balita: %.2f kg\n", rerata(data, n))
47 }

```

Output:

```

● PS D:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia> go run
  ramPosyandu.go"
Masukan banyak data berat balita: 4
Masukan berat balita ke-1: 5.3
Masukan berat balita ke-2: 6.2
Masukan berat balita ke-3: 4.1
Masukan berat balita ke-4: 9.9
Berat balita minimum: 4.10 kg
Berat balita maksimum: 9.90 kg
Rerata berat balita: 6.38 kg
○ PS D:\SEMESTER 2\ALPRO\PRAKTIKUM\109082530022_Haidar-Rafi-Arshia>

```

Penjelasan :

Program ini menyimpan hasil pertandingan menggunakan slice string yang berisi nama tim. Input dilakukan berulang hingga ditemukan nilai negatif sebagai kondisi berhenti. Logika percabangan digunakan untuk menentukan pemenang atau hasil seri, sesuai konsep array dan kontrol alur pada modul.

D. Kesimpulan

Berdasarkan Modul 10, kita dapat menyimpulkan bahwa mencari nilai ekstrim sebenarnya dilakukan satu per satu dengan membandingkan setiap elemen terhadap nilai acuan yang diperbarui terus-menerus. Cara ini tidak hanya berguna untuk mencari satu nilai pada tipe data dasar, tetapi juga sangat penting dalam mengenali objek pada tipe data terstruktur, seperti struct, melalui indeks penyimpanan. Agar algoritma ini berhasil, kita harus memastikan inisialisasi yang tepat pada elemen pertama dan melakukan validasi secara menyeluruh hingga akhir kumpulan data.